# Does Strict Decoding Recover Quantized Tool Calls? A Paired Small-Model Conceptual Replication

M. Mohith

## Abstract

A well-formed tool call can select the wrong function or value. We compare small Qwen and Llama models at full and quantized precision using a common GGUF engine and four decoder interventions: free generation, an unrestricted-text escape, a schema after call initiation, and whole-response enforcement. The frozen BFCL subset uses official AST and irrelevance scoring with paired item/run bootstrap intervals. Qwen3-0.6B has whole-strict incremental recovery of 2.1 [-0.7, 5.2] percentage points; Llama-3.2-1B has whole-strict incremental recovery of -4.7 [-9.1, -0.3] percentage points; Llama-3.2-3B has whole-strict incremental recovery of -2.1 [-5.0, 0.6] percentage points. These paired pointwise intervals are unadjusted and apply only to this setup. No primary free-decoding precision-loss interval excludes zero. We release responses, grammar checks, runtime probes, code and costs. This conceptual replication distinguishes structural guarantees from task accuracy and quantization-specific recovery.

## 1 Introduction

Local models must produce a supported representation and choose functions and arguments that answer the request. Weight compression can perturb any of these decisions. A schema removes invalid continuations but still admits incorrect actions: both Paris and London can satisfy a weather tool's city-string requirement.

TOOLDEC already enforces tool syntax without fine-tuning [17]. Prior work also separates structural and content failures and studies grammar mitigation in quantized models. Our question is narrower: does strict enforcement change the *incremental precision gap* differently from permissive or trigger-gated grammars? An intervention can improve a quantized model simply because it improves every precision. Conversely, a grammar with an arbitrary-text branch cannot establish a guarantee of schema-valid output. These observations motivate matched full-precision controls and an explicit check of the language each decoder admits.

We compare four interventions with paired uncertainty, retaining abstention, parallel calls and operational failures. This conceptual replication makes no priority claim about quantization damage, syntax repair or structural–semantic distinctions. The benchmark is single-turn prediction; no generated call is executed.

## 2 Related work and replication scope

Broad quantization studies emphasize dependence on the recipe and workload [8]. Agent diagnostics show that aggregate success can obscure process failures [7]; decision-level analyses already investigate which tool-use decisions low-bit weights disrupt [15]. These motivate a precision interaction, rather than a general claim that compressed models can make tool errors.

The closest software reports are QuantCall, QuantMCP and Toolhound [2, 3, 4]. Their inspected scopes differ in engine, precision controls, tasks and replication. QuantCall's default grammar unions a schema-shaped call with unrestricted text. QuantMCP describes a related escape and a forced-envelope probe with stalling. Toolhound reports trigger-gated comparisons on MLX and a custom suite. These are substantive prior experiments. Table 1 states which mechanisms we retest and what agreement would mean.

StructureBench and Chavan study structural validity and content accuracy with small models [16, 5]. Repair, Not Improvement isolates stopping and tool-name restrictions in a routing setting and already uses repeated runs and paired inference [9]. JSONSchemaBench emphasizes schema coverage and efficiency [6]. Constraint Tax highlights a separate hazard: a final-response grammar can exclude a serving protocol's tool calls [10]. We therefore constrain the declared call envelope and distinguish configuration, initiation, completion and correctness. Serving-stack differences can independently change apparent tool fidelity [14]. Our conceptual replication retests mechanisms, without reproducing another engine or live MCP task execution.

## 3 Experimental design

### 3.1 Frozen tasks, models and precision

We evaluate `Qwen3-0.6B`, `Llama-3.2-1B-Instruct`, `Llama-3.2-3B-Instruct`. The frozen sample contains 200 original cases: 60 simple, 60 multiple, 60 parallel,

Table 1: Conceptual replication targets. Agreement concerns mechanisms under changed conditions, not prior numerical scores. Empirical directional support requires a paired interval excluding zero; otherwise that contrast remains inconclusive.

| Prior work | What changes here | What would count as confirmation |
|---|---|---|
| QuantCall [2] | Shared BFCL subset, prompt/compiler, paired operational schedules; add strict enforcement. | A permissive escape gives little benefit while stricter enforcement changes validity or task accuracy. A null strict effect remains a valid outcome. |
| QuantMCP [3] | Replace live MCP execution with single-turn BFCL AST scoring; preserve abstention and log completion failures. | An escape can preserve malformed output; strict generation can still fail to terminate or answer correctly. No claim to reproduce MCP task-success rates. |
| Toolhound [4] | Replace MLX/custom tasks with llama.cpp/BFCL and add matched F16 controls. | Constraints improve call structure more consistently than task correctness. The added precision interaction need not match a q4-only gain. |

20 irrelevance. The primary required-call subset contains 180 items. Each precision–decoder cell has 5 operational runs, totaling 24,000 benchmark generations in the complete matrix. Decoding uses temperature 0.0, 4 parallel server slots, a 4096-token input-plus-output budget per request, and an output cap of 512 tokens. The same original items and accepted answers are retained across all conditions. Selection does not depend on baseline success, validity or a model prediction. The pilot subset was used for runtime and cost calibration, not to choose favorable tasks. Full-run prompts are frozen from publisher tokenizers and chat templates; the Llama template date is fixed. The common Qwen pilot prompts and whole-response grammars match the corresponding full-run inputs.

Weights are exported once to F16 GGUF and the quantized artifact is generated directly from that export. This reference is an F16 export, not the original checkpoint's possibly different storage dtype. Both K and V caches stay F16. Qwen thinking is disabled [13]. Model revisions, source/output hashes, conversion commands, server arguments and the pinned llama.cpp source [1] are recorded. All arms and precisions for a model use one A10 worker. The Modal allocation selector is fixed. Device-reported A10/A10G labels and drivers are retained rather than normalized away; model sessions need not share a physical host.

## 3.2 Decoder languages and prompt contract

Every arm for a given model sees exactly the same rendered prompt and offered tools. The primary Llama prompts use the single-object JSON tool branch of `tokenizer_config.json`'s embedded HF chat template, with `tools_in_user_message=True`. Both selected revisions have the same template hash, recorded in Appendix A. We added a JSON-array convention for parallel calls in the system message, while that HF template inserts a later user-message instruction requesting one JSON object. These instructions conflict on parallel tasks. Meta separately documents a Pythonic list of calls [11]; our primary JSON contract does not evaluate that format. Llama findings are conditional on this declared contract, rather than claims about its best supported interface. Qwen uses tagged call objects with `name` and `arguments`; Llama uses objects with `name` and `parameters`, or arrays for parallel calls. All arms use `NO_CALL` when no offered tool applies. The grammar never uses the expected function, accepted values or gold call count.

**Free** applies no grammar. **Loose** unions the schema-shaped language with arbitrary text, reproducing the escape as a control. **Trigger-strict** leaves initiation unconstrained and activates the call language at the declared prefix. Qwen uses its preserved call token; Llama uses one pattern matching the earliest object or array opener, including an opener merged into a token. **Whole strict** admits only a documented call sequence or the explicit no-call form. Parallel-call count and order are not fixed to the reference. Lazy activation can encounter a merged trigger-containing token whose remainder violates the call language. Known replay errors are retained as failed attempts, rather than retried or admitted through a grammar escape.

The compiler permits arbitrary ordering of documented object keys while enforcing required/unique keys. Known property sets are closed as an explicit API policy. Free-form dictionaries remain free-form. A nested BFCL dictionary that names required fields without field types retains those names with unconstrained values; types are not guessed from answers. These normalization choices are part of the intervention and are released.

Finite character-level membership checks cover the selected schemas, malformed strings, unknown functions, missing/extra arguments, types, key order, parallel calls and abstention. They are evidence of tested coverage, not a proof of every JSON Schema feature or token-level behavior. Separate synthetic runtime probes force a known trigger and verify literal replay and the server's configured lazy mode before benchmark collection. These probes are setup, not benchmark successes. No engine patch is used.

### 3.3 Execution and scoring

Temperature is zero, with seed/schedule labels frozen before execution and shuffled requests. Identical greedy outputs are retained as operational repeats; they are not new independent items. Configured context/output caps, server slots, cache settings and stop strings match across arms. Grammar acceptance itself changes when termination is legal. Llama prompts use publisher token IDs to avoid duplicating the beginning-of-text marker already present in its template. An earlier collection with that duplication is retained as an aborted setup attempt, before task-accuracy inspection. Qwen uses its original string transport with publisher settings that disable added BOS. Cache-reuse timings and actual generation settings are logged. The cloud supervisor checkpoints events, enforces queue/runtime bounds and enforces the scheduling rule frozen before execution. The researcher permitted concurrent full studies after the separate pilots; this does not establish isolated cross-model throughput.

Official BFCL AST matching is the correctness authority [12]. We adapt the declared JSON envelope into calls without executing outputs. The pinned checker is unmodified; a minimal registry entry supplies its name-normalization flag without importing unrelated API clients. The unchanged official irrelevance helper is used through the same parsing adapter. Irrelevance can credit an output with no parsed call while our format diagnostic rejects its prose; those outcomes are reported separately. Required-call accuracy is the primary endpoint.

Our deterministic diagnostics retain multiple flags and an ordered primary label: format failure, wrong or hallucinated function, missing/extra or hallucinated arguments, wrong type and wrong value, plus missing/extra calls and operational statuses. A schema-valid parallel mismatch is not labeled a known value error without evidence from the checker. Incomplete output and runtime failures remain visible. Unstarted budget-censored cells are reported as incomplete, rather than filled with synthetic predictions.

### 3.4 Estimands and paired uncertainty

Let $A_{p,d}$ be official item accuracy at precision $p$ and decoder $d$, with $f$ denoting free decoding. For the quantized precision $q$, define

$$L_f = A_{\mathrm{F16},f} - A_{q,f}, \tag{1}$$

$$L_d = A_{\mathrm{F16},d} - A_{q,d}, \tag{2}$$

$$R_d = L_f - L_d. \tag{3}$$

$R_d$ removes the decoder effect already present at F16. We also compare $R_{\mathrm{whole}} - R_{\mathrm{loose}}$ and the corresponding trigger contrast on paired observations. These are intervention effects, not an identified fraction of purely syntactic knowledge loss: schemas restrict names, keys and types, and can redirect subsequent value choices.

Primary percentile intervals use crossed paired resampling of original item IDs and operational-run labels, sharing draws across all conditions and models. Item-only intervals are saved as a sensitivity check. Category rates use their original item subsets. Intervals are pointwise and unadjusted for multiple comparisons; exploratory contrasts are labeled accordingly. At a boundary with no observed failures, empirical bootstrap intervals can degenerate; this is not a bound on rare unobserved population failures. A recovery-share point estimate $R_d/L_f$ is undefined when observed free loss is nonpositive. Its interval is withheld when bootstrap denominators are unstable; values are not clipped into a manufactured partition.

## 4 Results

Qwen3-0.6B: Q4 required-call accuracy changes from 70.7% free to 75.6% whole strict. Its incremental recovery is 2.1 [-0.7, 5.2] percentage points. Across all cases for Qwen3-0.6B, Q4 whole-strict response-structure validity is 100.0 [100.0, 100.0]%, while official accuracy is 73.0 [66.9, 79.0]%.

Llama-3.2-1B: Q4 required-call accuracy changes from 14.2% free to 52.7% whole strict. Its incremental recovery is -4.7 [-9.1, -0.3] percentage points. Across all cases for Llama-3.2-1B, Q4 whole-strict response-structure validity is 100.0 [100.0, 100.0]%, while official accuracy is 47.4 [40.5, 54.2]%. Its Q4 whole-strict parallel accuracy under the declared JSON contract is 1.0 [0.0, 4.0]%. This does not measure the publisher Pythonic call-list contract.

Llama-3.2-3B: Q4 required-call accuracy changes from 29.2% free to 61.0% whole strict. Its incremental recovery is -2.1 [-5.0, 0.6] percentage points. Across all cases for Llama-3.2-3B, Q4 whole-strict response-structure validity is 100.0 [100.0, 100.0]%, while official accuracy is 54.9 [48.0, 61.7]%. Its Q4 whole-strict parallel accuracy under the declared JSON contract is 0.0 [0.0, 0.0]%. This does not measure the publisher Pythonic call-list contract.

Generation status counts are grammar_trigger_replay_error: 1; ok: 23860; token_limit: 139. Negative incremental recovery does not mean constraints lowered Q4 accuracy: they helped F16 more in these cells. Valid structure includes no-call responses; task correctness and irrelevance are scored separately. Other diagnostic rates, category endpoints, item-only sensitivity intervals, paired decoder contrasts and raw responses are released with the result tables. Intervals crossing zero do not establish equivalence.

**Separate Pythonic-format sensitivity.** We additionally collected 4,800 responses on 120 frozen cases, using the original parallel subset plus a separately sampled parallel-multiple subset, both Llama models, both precisions, free/whole strict and the same operational labels. The prompt follows Meta call-list syntax, omits the

Table 2: Required-call accuracy (%) and crossed paired 95% intervals. Repeated schedules remain clustered with original items.

| Model | Precision | Free | Loose | Triggered | Whole strict |
|---|---|---|---|---|---|
| Qwen3-0.6B | F16 | 73.9 [67.2, 80.0] | 73.9 [67.2, 80.0] | 76.7 [70.0, 82.8] | 76.7 [70.0, 82.8] |
| Qwen3-0.6B | Q4_K_M | 70.7 [64.0, 77.0] | 70.7 [63.9, 77.0] | 74.8 [68.4, 80.7] | 75.6 [69.2, 81.4] |
| Llama-3.2-1B | F16 | 12.8 [8.3, 17.8] | 12.8 [8.3, 17.8] | 54.3 [47.1, 61.7] | 55.9 [48.7, 63.1] |
| Llama-3.2-1B | Q4_K_M | 14.2 [9.3, 19.4] | 14.1 [9.2, 19.4] | 51.9 [44.7, 59.1] | 52.7 [45.3, 59.9] |
| Llama-3.2-3B | F16 | 27.2 [20.6, 33.9] | 27.2 [20.6, 33.9] | 61.1 [53.9, 68.3] | 61.1 [53.9, 68.3] |
| Llama-3.2-3B | Q4_K_M | 29.2 [22.7, 35.9] | 29.4 [22.8, 36.1] | 60.4 [53.3, 67.6] | 61.0 [53.9, 68.1] |

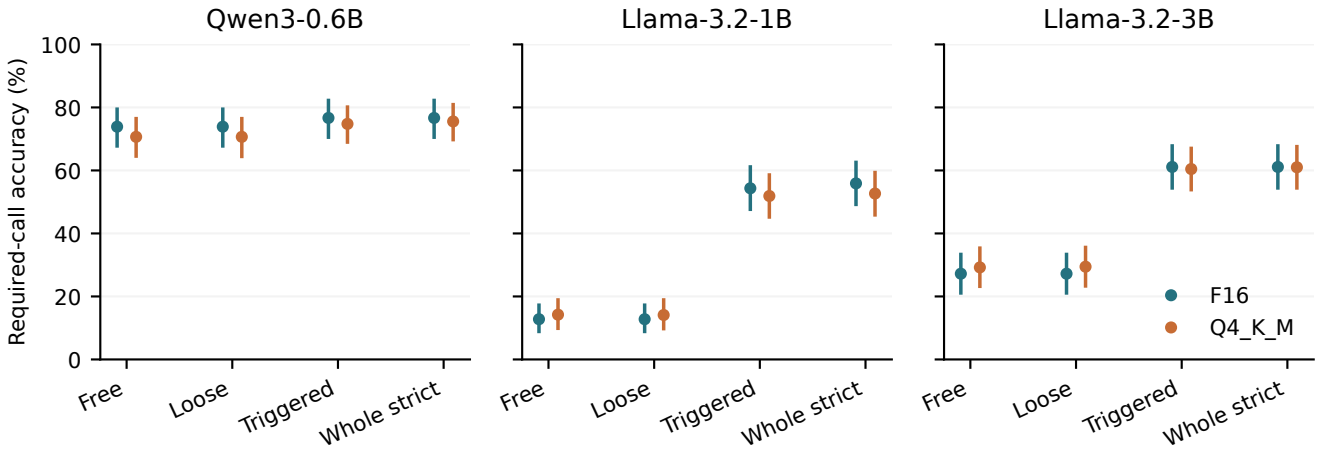



Figure 1: Required-call accuracy under each intervention. Whiskers are crossed paired bootstrap intervals, not independent-binomial intervals over repeated generations.

HF tools injection, and uses [] for abstention. Llama-3.2-1B Q4 whole-strict parallel accuracy is 62.3 [50.0, 74.3]%, versus 1.0% under the original JSON contract. Its Pythonic parallel incremental recovery is 1.7 [-10.3, 13.7] points; the paired change in recovery versus JSON is 2.3 [-10.0, 15.0] points; Llama-3.2-3B Q4 whole-strict parallel accuracy is 88.3 [80.0, 95.7]%, versus 0.0% under the original JSON contract. Its Pythonic parallel incremental recovery is -3.3 [-8.7, 0.7] points; the paired change in recovery versus JSON is -3.3 [-8.7, 0.7] points. These gains change the interpretation of the low primary Llama parallel scores: they are strongly contract-dependent, rather than evidence of an intrinsic inability to make parallel calls. Neither parallel incremental recovery nor its protocol change is resolved from zero. This is a joint prompt/parser/grammar sensitivity, not a parser-only causal estimate. Only the original parallel items have a matched primary baseline; parallel-multiple results are reported separately. Sessions differ in time and host, so this is not a randomized crossover. Llama-3B was interrupted by the platform after 1,610 durable outputs; only the missing 790 cells continued on another A10, with saved failures retained. The primary matrix is unchanged. Additional status counts are ok: 4222; token_limit: 578. These limits remain incorrect; no added exclusion analysis is claimed.

# 5 Cost, limitations and practical implications

The pilot planning scenario for the primary campaign was $4.74–$8.14. Gross tagged project cost reported at the pre-follow-up snapshot is $2.6110, including retained setup and interrupted attempts, against the $12.00 cap. Metering is reported to date, not a settled invoice. All final app and container states are saved separately from the billing snapshot. The separate Pythonic sensitivity adds $0.7346, including failed/interrupted apps, for latest reported project cost $3.3456 against the additional $3.00 and total $12.00 caps. Forecasts are assumptions rather than observations. Timed estimates omit untimed platform stages; metering can lag and does not establish storage completeness. Earlier failures remain in the release.

This small frozen BFCL sample is not a complete leaderboard evaluation. AST correctness does not establish safe execution, successful multi-turn agents, or correctness on unseen APIs. Irrelevance has a small diagnostic sample. The inference stack, cache dtype, weight recipe, output envelope and schema normalization are fixed; results do not automatically extend to other quantizers, KV-cache compression, larger catalogs or different engines. Prefix triggering is an operational rule, not a direct measure of internal intent. Log-derived activation

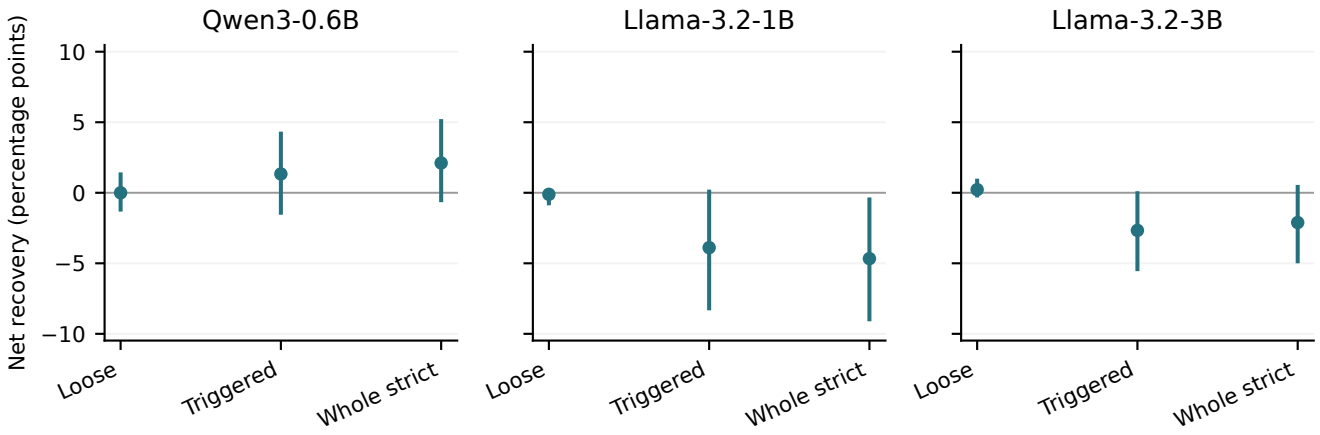



Figure 2: Signed recovery of the incremental F16–Q4 gap. Zero means no observed reduction of that gap; negative effects are retained. This quantity does not causally separate syntax from semantics.

Table 3: Required-call signed precision loss and incremental recovery, in percentage points [95% CI]. The final column is the normalized share (%) when its denominator is stable.

| Model | Decoder | Free loss | Constrained loss | Net recovery | Recovery share |
|---|---|---|---|---|---|
| Qwen3-0.6B | Loose | 3.2 [-1.9, 8.3] | 3.2 [-1.9, 8.3] | 0.0 [-1.3, 1.4] | unstable |
| Qwen3-0.6B | Triggered | 3.2 [-1.9, 8.3] | 1.9 [-2.4, 6.2] | 1.3 [-1.6, 4.3] | unstable |
| Qwen3-0.6B | Whole strict | 3.2 [-1.9, 8.3] | 1.1 [-3.2, 5.3] | 2.1 [-0.7, 5.2] | unstable |
| Llama-3.2-1B | Loose | -1.4 [-4.4, 1.4] | -1.3 [-4.4, 1.6] | -0.1 [-0.9, 0.3] | undefined |
| Llama-3.2-1B | Triggered | -1.4 [-4.4, 1.4] | 2.4 [-1.8, 6.7] | -3.9 [-8.3, 0.2] | undefined |
| Llama-3.2-1B | Whole strict | -1.4 [-4.4, 1.4] | 3.2 [-0.7, 7.2] | -4.7 [-9.1, -0.3] | undefined |
| Llama-3.2-3B | Loose | -2.0 [-4.6, 0.3] | -2.2 [-4.9, 0.2] | 0.2 [-0.3, 1.0] | undefined |
| Llama-3.2-3B | Triggered | -2.0 [-4.6, 0.3] | 0.7 [-0.8, 2.2] | -2.7 [-5.6, 0.1] | undefined |
| Llama-3.2-3B | Whole strict | -2.0 [-4.6, 0.3] | 0.1 [-1.3, 1.4] | -2.1 [-5.0, 0.6] | undefined |

is distinguished from a direct trace of sampler state.

**Primary failures and post-hoc exclusion.** Qwen3-0.6B has 0 token-limit outcomes and 0 replay errors; Llama-3.2-1B has 101 token-limit outcomes and 1 replay error; Llama-3.2-3B has 38 token-limit outcomes and 0 replay errors. Failures concentrate in particular decoder/task cells; exclusion cannot be assumed independent of treatment or difficulty. The analysis frozen before execution counts them as incorrect. We additionally exclude failures within each cell, exclude a model/item/run from all precision–decoder cells whenever any cell failed, and exclude whole items with any failure across conditions/runs. The middle rule preserves a common observed cohort; its required-call whole-strict recovery is Qwen3-0.6B 2.1 [-0.7, 5.2]; Llama-3.2-1B -4.8 [-9.4, -0.5]; Llama-3.2-3B -2.1 [-5.1, 0.6] percentage points [95% CI]. Directional interval classifications change in 0 recovery effects and 0 decoder contrasts across all three rules and both endpoints. Cell-only exclusion compares different completed populations; every exclusion selects on a post-treatment outcome and can favor easier items. None identifies performance with unlimited generation or a repaired engine. The released failure report includes every cell count, retained denominator and supplementary interval; primary results remain unchanged.

Closed schemas can forbid useful undocumented arguments; typed allowed values can still be wrong. Parser policy affects prose, abstention and parallel-call scoring. Operational repeats do not resample training runs, quantization builds or templates, or establish generalization across hardware. Intervals crossing zero do not prove equality.

Operators need task accuracy under the same serving contract and a matched high-precision control. Structural guarantees remain useful without accuracy gains. The planned Direction 2 follow-up compares schema-audit/precision fallback with ordinary retries; it is not evaluated here.

# 6 Conclusion

This conceptual replication compares constraints under the declared serving contracts. Its pointwise, unadjusted intervals apply to this setup only. No primary free-decoding precision-loss interval excludes zero. Qwen3-0.6B's observed whole-strict incremental recovery is not resolved from zero by its interval. Llama-3.2-1B's observed whole-strict incremental recovery is negative. Llama-3.2-3B's observed whole-strict incremental recovery is not resolved from zero by its interval. Structural recovery and the weight-precision interaction must there-

fore be reported separately; a raw constrained gain alone cannot identify recovered quantization damage.

# A Settings and availability

Code, frozen data, responses, analysis and the paper are at `https://github.com/mohi-devhub/quntic-check`. The primary campaign is `full-002`; the Pythonic sensitivity is separate. The design was frozen before execution, without a public preregistration timestamp. The code is MIT licensed; redistributed BFCL material includes Gorilla's Apache-2.0 license and attribution.

BFCL v4 data and checker are pinned to Gorilla revision `6ea57973c7a6097fd7c5915698c54c17c5b1b6c8`. The engine is llama.cpp revision `8542e94350f0ade01644db6f1d1520da3814dafd`. Crossed bootstrap intervals use 10,000 draws and seed 20261028. Operational labels are 101, 211, 307, 401 and 503. Qwen uses `enable_thinking=False`. Each model loads F16 first and Q4 second, with the five shuffled schedules inside each precision block; precision order was not counterbalanced. These intervals describe item/schedule variation under that execution.

The Llama HF JSON templates are from 1B revision `9213176726f574b556790deb65791e0c5aa438b6` and 3B revision `0cb88a4f764b7a12671c53f0838cd831a0843b95`. Their template SHA-256 is `2732d9da5568668ed22ea202bffb12f0df405439651e1275ba1f521522f92966`. Exact executed worker sources and hash records are in `provenance/`; the maintained implementation is separate. Historical launcher revisions were not retained, so we do not claim to have recovered them. Prompts and per-run commands are released.

# References


[1] llama.cpp. Pinned software. `https://github.com/ggml-org/llama.cpp/tree/8542e94350f0ade01644db6f1d1520da3814dafd`.

[2] Quantcall. Pinned software and experiment reports. `https://github.com/Happynood/quant-toolcall-bench/tree/6b6e29e5c83a91a52bdfc68b5f3063de172c0e55`.

[3] Quantmcp. Pinned software and experiment reports. `https://github.com/Happynood/quant-mcp-bench/tree/fc98a47c6a0ef77bb8be3a915016b42d203faf45`.

[4] Toolhound. Pinned software and experiment reports. `https://github.com/Code-byte404/toolhound/tree/97666be79a2af2afb78b36a168a5f89f84be6d97`.

[5] Akash Chavan. Constrained decoding eliminates structural failures in small llms but reveals a scale-dependent semantic gap. arXiv:2609.23742v1, 2026. `https://arxiv.org/abs/2609.23742v1`.

[6] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. Jsonschemabench: A rigorous benchmark of structured outputs for language models. arXiv:2501.10868v3, 2025. `https://arxiv.org/abs/2501.10868v3`.

[7] Jiwon Jang, Kisu Yang, Heuiseok Lim, and Hyunwoo Park. Flat score, amplified failures: How the error budget masks damage in quantized llm agents. arXiv:2607.27275v1, 2026. `https://arxiv.org/abs/2607.27275v1`.

[8] Eldar Kurtic, Alexandre Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. "Give Me BF16 or Give Me Death"? Accuracy-Performance Trade-Offs in LLM Quantization. arXiv:2411.02355v4, 2024. `https://arxiv.org/abs/2411.02355v4`.

[9] Janghoon Lee. Repair, not improvement: Decomposing constrained decoding in tool-call abstention. arXiv:2608.13959v2, 2026. `https://arxiv.org/abs/2608.13959v2`.

[10] Fangzheng Li, Aimin Zhang, and Chen Lv. Constraint tax in open-weight llms: An empirical study of tool calling suppression under structured output constraints. arXiv:2606.25605v1, 2026. `https://arxiv.org/abs/2606.25605v1`.

[11] Meta. Llama 3.2 text prompt format. `https://github.com/meta-llama/llama-models/blob/0e0b8c519242d5833d8c11bffc1232b77ad7f301/models/llama3_2/text_prompt_format.md`.

[12] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. Proceedings of Machine Learning Research, volume 267, 2025. `https://proceedings.mlr.press/v267/patil25a.html`.

[13] Qwen Team. Qwen3-0.6B model card, 2025. `https://huggingface.co/Qwen/Qwen3-0.6B/tree/c1899de289a04d12100db370d81485cdf75e47ca`.

[14] Lijuan Tang and Yuemeng Zheng. Measuring the serving stack instead of the model: Hidden confounds in local tool-use evaluation. arXiv:2609.26693v1, 2026. `https://arxiv.org/abs/2609.26693v1`.

[15] Zekun Wu, Swati Dhiman, and Adriano Koshiyama. Which decisions low-bit quantization breaks, and how to predict them. arXiv:2608.06564v5, 2026. `https://arxiv.org/abs/2608.06564v5`.

[16] Xiaokun Xiong, Zhengjie Xu, Junyi Chen, Shihao Bai, Ruihao Gong, and Xianglong Liu. Structurebench: A unified benchmark suite for multi-scenario structured generation tasks with on-device models. IJCAI proceedings, 2026. `https://ijcai.org/proceedings/2026/267`.

[17] Kexun Zhang, Hongqiao Chen, Lei Li, and William Wang. Don't fine-tune, decode: Syntax error-free tool use via constrained decoding. arXiv:2310.07075v3, 2024. `https://arxiv.org/abs/2310.07075v3`.